

reborn: A faithful R port of seaborn built on the grammar of graphics

Shawn T. Schwartz*

Department of Psychology, Stanford University, United States

June 24, 2026

Abstract

reborn is an R package that ports the public function interface of the Python visualization library seaborn onto ggplot2, the dominant grammar-of-graphics engine for R. It mirrors seaborn’s roughly forty plotting functions—their names, arguments, and default values—so that a chart written for seaborn produces a visually equivalent figure in R, and it reproduces seaborn’s underlying numerics (kernel density estimates, histogram binning, bootstrap confidence intervals, and color palettes) to matching results. Unlike seaborn, whose author describes it as distinct from the grammar of graphics, every reborn function returns an ordinary ggplot2 object. reborn therefore unifies two approaches that have historically been treated as separate: the rapid, opinionated, named-function workflow of seaborn and the composable layered grammar of ggplot2. Users obtain sensible, publication-ready scaffolding from a single call, then refine the result with the same ggplot2 grammar they already know.

1 Summary

reborn ports the public application programming interface (API) of seaborn (Waskom, 2021) onto ggplot2 (Wickham, 2016), the dominant grammar-of-graphics engine for R (R Core Team, 2025). It mirrors seaborn’s roughly forty plotting functions—their names, argument names, and default values—so that a chart specification written for seaborn produces a visually equivalent figure in R. Attaching the package with `library(reborn)` installs seaborn’s theme and color palette globally (as `sns.set_theme()` does in Python), exposes `sns.`-prefixed aliases for every function, and binds the Python literals `True`, `False`, and `None`. Consequently, the call

```
library(reborn)
penguins <- load_dataset("penguins")
sns.scatterplot(data = penguins, x = "bill_length_mm",
                y = "bill_depth_mm", hue = "species")
```

runs verbatim in R. Crucially, every reborn function returns an ordinary ggplot2 object, so the resulting figure can be refined with the full grammar of graphics using vocabulary that ggplot2 users already know.

*Corresponding author: shawn.t.schwartz@gmail.com. ORCID: [0000-0001-6444-8451](https://orcid.org/0000-0001-6444-8451).

2 Statement of need

Data visualization is integral to the scientific process, serving both to help a researcher understand their own data and to communicate findings to others. The R ecosystem provides this capability primarily through `ggplot2`, an implementation of the layered grammar of graphics (Wickham, 2010) that formalizes Wilkinson’s *Grammar of Graphics* (Wilkinson, 2005). The grammar is expressive but deliberately low-level: producing a polished exploratory figure typically requires several layers together with a block of theme and scale code, which many analysts copy into the top of every notebook or project as boilerplate.

In the Python ecosystem, `seaborn` addresses this friction with opinionated, named functions that infer sensible defaults—semantic colour, size, and style mappings; statistical transformations with bootstrap confidence intervals; faceting; and a coordinated visual theme—from a single call (Waskom, 2021). No native R package reproduces this interface. Analysts who know `seaborn` and move to R must re-learn a different idiom, and `ggplot2` users who want `seaborn`’s defaults must reconstruct them by hand. The only way to obtain *actual* `seaborn` output in R is through `reticulate` (Ushey et al., 2025), which requires a Python and `seaborn` installation and returns static `matplotlib` raster images rather than composable `ggplot2` objects. `reborn` removes this friction: it gives `seaborn` users a zero-translation landing in R and gives `ggplot2` users `seaborn`-grade scaffolding that remains fully manipulable with the grammar they already use.

3 Unifying opinionated defaults and the grammar of graphics

`reborn`’s central contribution is that it dissolves a dichotomy the `seaborn` paper draws explicitly. Waskom (Waskom, 2021) states that `seaborn` “does not implement the formal Grammar of Graphics and cannot be used to produce arbitrary visualizations,” instead offering rapid exploration through opinionated named functions while deferring deeper customization to `matplotlib`. `seaborn`’s axes-level functions return a `matplotlib Axes`, which is mutated in place rather than composed; further customization happens by passing keyword arguments through to `matplotlib` or by calling methods on the returned object.

In `reborn`, the same opinionated, named-function workflow yields a value that *is* a `ggplot2` object. The two paradigms therefore coexist in one expression:

```
scatterplot(data = penguins, x = "bill_length_mm",
            y = "bill_depth_mm", hue = "species") +
  ggplot2::facet_wrap(~island) +
  ggplot2::scale_x_log10() +
  ggplot2::theme(legend.position = "top")
```

A user obtains `seaborn`’s sensible defaults from the first call, then refines the result with arbitrary `ggplot2` layers—facets, scales, additional geoms, themes—using grammar they already know, without writing the theme and scale boilerplate that `ggplot2` figures usually accumulate. Figure 2 demonstrates this composition. We note that this is the `seaborn` author’s own framing; `seaborn`’s newer `objects` interface adds grammar-like composition in Python. `reborn`’s specific contribution is to deliver the named-function-defaults API *returning grammar-composable objects* natively in R.

4 Software design and statistical fidelity

`reborn` is not a thin wrapper. Its plotting functions funnel through a shared data-assignment engine that resolves long- and wide-form input (Wickham, 2014) and a semantic-mapping layer that

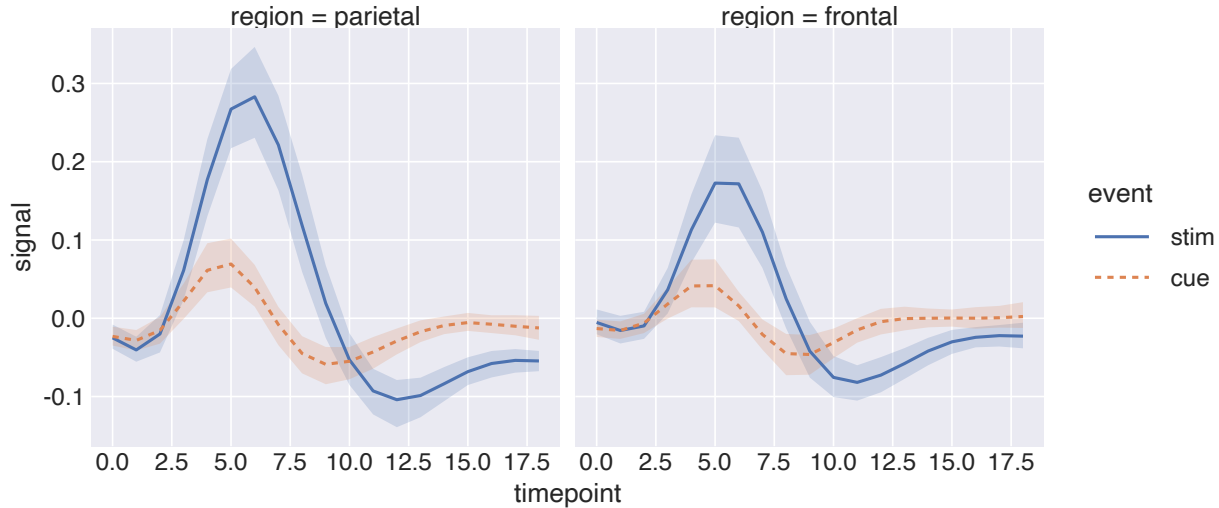


Figure 1: The `fmri relplot` example from the seaborn documentation, reproduced in reaborn with the identical call. The style mapping, bootstrap confidence bands, faceting, and theme are reproduced natively in R.

translates seaborn’s `hue`, `size`, and `style` arguments into ggplot2 scales. To make figures genuinely indistinguishable from seaborn rather than merely similar, the package reproduces seaborn’s underlying numerics: kernel density estimates match `scipy.stats.gaussian_kde` (Virtanen et al., 2020) to machine precision; histogram bins reproduce `numpy.histogram_bin_edges` (Harris et al., 2020) exactly; error bars use seaborn’s bootstrap confidence intervals rather than ggplot2’s analytic standard errors; and the colour palettes, including matplotlib’s colormap quantization and named-colour table (Hunter, 2007), are matched to identical hexadecimal codes. Custom statistics that ggplot2 does not ship—the kernel-density violin, the letter-value (boxen) plot, and the beeswarm layout—are implemented directly, and multi-panel grids (`pairplot`, `jointplot`) are assembled with `patchwork` (Pedersen, 2025). Each design choice reflects an explicit trade-off between fidelity to seaborn and idiomatic R; where the two conflict, reaborn preserves the behaviour a seaborn user would expect while keeping the return value a standard ggplot2 object.

5 Example

Figure 1 reproduces the `fmri relplot` example from the seaborn documentation using the identical call, demonstrating native support for the declarative API, semantic mappings (the `style` variable becomes a dashed line), bootstrap aggregation with error bands, faceting across subplots, and theme control. Figure 2 demonstrates the unification thesis: a one-line reaborn call extended with ggplot2’s `stat_ellipse()` and `facet_wrap()`.

6 State of the field

Because ggplot2 is a low-level grammar, a rich ecosystem of higher-level helpers has grown around it to supply opinionated defaults and reduce boilerplate: `ggpubr` wraps ggplot2 into publication-ready calls with statistical annotations (Kassambara, 2026), `GGally` adds pairs and correlation matrices via `ggpairs` (Schloerke et al., 2025), `ggdist` and the `easystats` `see` package add distribution and

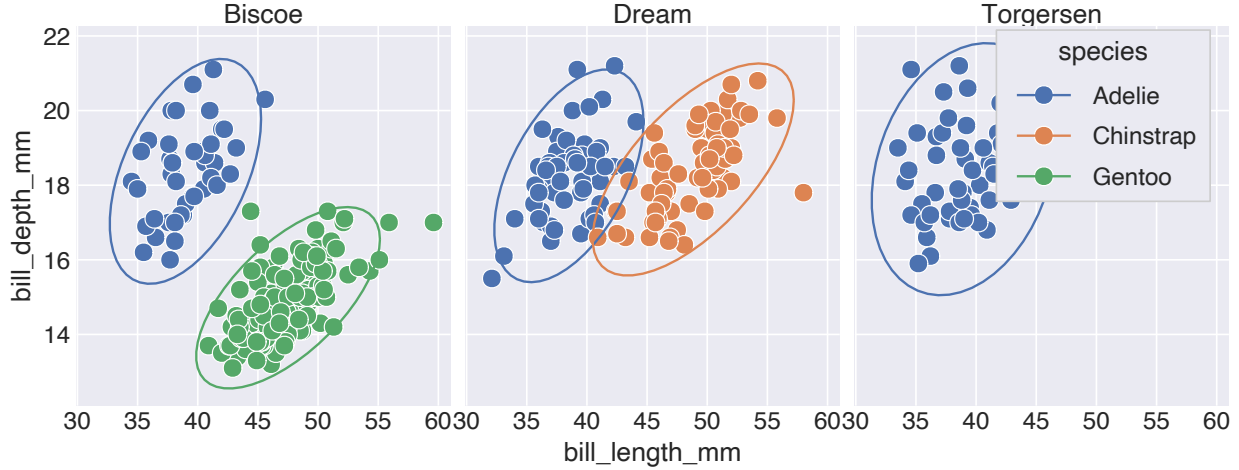


Figure 2: A reaborn one-liner returns a ggplot2 object, so native grammar-of-graphics layers compose directly onto it. Here a `scatterplot()` call is extended with ggplot2’s `stat_ellipse()` and `facet_wrap(~island)`.

model-visualization geoms (Kay, 2024; Lüdecke et al., 2021), cowplot and patchwork compose multi-panel figures (Wilke, 2024; Pedersen, 2025), and lattice offers an alternative trellis system (Sarkar, 2008). Each of these is built on, and returns, ggplot2 or grid objects, but none reproduces seaborn’s public API—its named functions, argument names, defaults, and specific statistics. The reverse bridge, calling seaborn from R via reticulate (Ushey et al., 2025), requires a Python runtime and yields static matplotlib raster output rather than grammar-composable ggplot2 objects. The closest conceptual precedent runs in the opposite direction: plotnine, a port of ggplot2 to Python (Kibirige et al., 2025). reaborn fills the symmetric and, to our knowledge, first faithful seaborn-API port to R whose every plot is a ggplot2 object, thereby unifying the opinionated-defaults and grammar-of-graphics approaches that seaborn’s author described as separate.

7 Availability, quality, and reproducibility

reaborn is released under the BSD 3-Clause licence (the same licence as seaborn) and is developed openly at <https://github.com/shawntz/reaborn>, with full documentation, tutorials, and a function reference at <https://reaborn.org>. The package is feature-complete across seaborn’s relational, distributional, categorical, regression, matrix, and multi-panel families. Quality is maintained through continuous integration across platforms, a clean R CMD check, and an extensive automated test suite. A distinguishing element of the test suite is a *fidelity* component that verifies, against the reference implementations, that palettes, kernel density estimates, histogram bin edges, and bootstrap intervals match seaborn—a reproducible benchmark of cross-language equivalence rather than an aspirational claim. A versioned software archive is available [via the repository](#).

8 AI usage disclosure

Generative AI coding assistants were used to assist with portions of the implementation, test scaffolding, documentation, and the drafting of this manuscript. All AI-generated output was reviewed, edited, and validated by the author; the core architectural and design decisions were

made by the author, and all statistical-fidelity claims were verified against the seaborn, SciPy, and NumPy reference implementations.

Acknowledgements

We thank the developers of seaborn, ggplot2, and the broader matplotlib and tidyverse ecosystems, upon whose work reaborn builds.

References

- Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Pícus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020. doi: 10.1038/s41586-020-2649-2. URL <https://doi.org/10.1038/s41586-020-2649-2>.
- John D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007. doi: 10.1109/MCSE.2007.55. URL <https://doi.org/10.1109/MCSE.2007.55>.
- Alboukadel Kassambara. *ggpubr: 'ggplot2' Based Publication Ready Plots*, 2026. URL <https://CRAN.R-project.org/package=ggpubr>. R package version 0.6.3.
- Matthew Kay. ggdist: Visualizations of distributions and uncertainty in the grammar of graphics, 2024. R package.
- Hassan Kibirige, Greg Lamp, Jan Katins, Florian Finkernagel, Tyler Funnell, Jonas Arnfred, Dan Blanchard, Paul Natsuo Kishimoto, Sergey Astanin, Eric Chiang, Evan Sheehan, Bernard Willers, Michael Chow, Jeroen Janssens, Simon Mutch, et al. plotnine: A grammar of graphics for Python, 2025. URL <https://github.com/has2k1/plotnine>. Python package.
- Daniel Lüdtke, Indrajeet Patil, Mattan S. Ben-Shachar, Brenton M. Wiernik, Philip Waggoner, and Dominique Makowski. see: An R package for visualizing statistical models. *Journal of Open Source Software*, 6(64):3393, 2021. doi: 10.21105/joss.03393.
- Thomas Lin Pedersen. *patchwork: The Composer of Plots*, 2025. URL <https://CRAN.R-project.org/package=patchwork>. R package version 1.3.2.
- R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2025. URL <https://www.R-project.org/>.
- Deepayan Sarkar. *Lattice: Multivariate Data Visualization with R*. Springer, New York, 2008. ISBN 978-0-387-75968-5. doi: 10.1007/978-0-387-75969-2.
- Barret Schloerke, Di Cook, Joseph Larmarange, Francois Briatte, Moritz Marbach, Edwin Thoen, Amos Elberg, and Jason Crowley. *GGally: Extension to 'ggplot2'*, 2025. URL <https://CRAN.R-project.org/package=GGally>. R package version 2.4.0.
- Kevin Ushey, JJ Allaire, and Yuan Tang. *reticulate: Interface to 'Python'*, 2025. URL <https://CRAN.R-project.org/package=reticulate>. R package version 1.44.1.

- Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C. J. Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3):261–272, 2020. doi: 10.1038/s41592-019-0686-2. URL <https://doi.org/10.1038/s41592-019-0686-2>.
- Michael L. Waskom. seaborn: statistical data visualization. *Journal of Open Source Software*, 6(60): 3021, 2021. doi: 10.21105/joss.03021. URL <https://doi.org/10.21105/joss.03021>.
- Hadley Wickham. A layered grammar of graphics. *Journal of Computational and Graphical Statistics*, 19(1):3–28, 2010. doi: 10.1198/jcgs.2009.07098.
- Hadley Wickham. Tidy data. *Journal of Statistical Software*, 59(10):1–23, 2014. doi: 10.18637/jss.v059.i10. URL <https://doi.org/10.18637/jss.v059.i10>.
- Hadley Wickham. *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York, 2nd edition, 2016. ISBN 978-3-319-24277-4. doi: 10.1007/978-3-319-24277-4. URL <https://ggplot2.tidyverse.org>.
- Claus O. Wilke. *cowplot: Streamlined Plot Theme and Plot Annotations for ggplot2*, 2024. URL <https://CRAN.R-project.org/package=cowplot>. R package.
- Leland Wilkinson. *The Grammar of Graphics*. Statistics and Computing. Springer-Verlag New York, 2nd edition, 2005. ISBN 978-0-387-24544-7. doi: 10.1007/0-387-28695-0. URL <https://doi.org/10.1007/0-387-28695-0>.